

Введение

В данной работе было выполнено развитие предыдущих практических заданий, связанных с контейнеризацией и развертыванием веб-приложения в Kubernetes. Если в прошлых этапах приложение разворачивалось с помощью обычных Kubernetes-манифестов, то целью данной работы стал переход на более удобный и масштабируемый способ управления развертыванием с использованием Helm.

Основная задача заключалась в переработке существующей структуры развертывания таким образом, чтобы приложение и сопутствующие инфраструктурные компоненты устанавливались через Helm-чарты. Дополнительно требовалось внедрить современные Kubernetes-решения для маршрутизации, управления базой данных и выпуска сертификатов. В рамках работы `Gateway API` также был переведен на Helm-установку, самостоятельно развернутый PostgreSQL был заменен на оператор `CloudNativePG`, а для настройки защищенного HTTPS-доступа были добавлены самоподписанные сертификаты с использованием оператора `cert-manager`.

В качестве целевого приложения использовался веб-сервис, состоящий из frontend- и backend-частей. Frontend предоставляет пользовательский интерфейс для просмотра списка людей, а backend реализует API и взаимодействует с базой данных PostgreSQL. После переработки проект получил новую структуру, в которой отдельно выделены Helm-чарты для платформенных компонентов и для самого приложения.

В процессе выполнения работы были настроены и установлены следующие ключевые элементы:

- Helm-чарты для platform- и application-уровня;
- `NGINX Gateway Fabric` и `Gateway API` для маршрутизации трафика;
- оператор `CloudNativePG` для управления PostgreSQL в Kubernetes;
- оператор `cert-manager` для выпуска и управления TLS-сертификатами;
- защищенный HTTPS-доступ к приложению с использованием self-signed сертификатов.

Результатом работы стало полностью функционирующее приложение, развернутое в Kubernetes с использованием Helm, доступное через `Gateway API` по HTTPS и использующее PostgreSQL, управляемый оператором `CloudNativePG`.

Анализ исходного проекта и подготовка к миграции на Helm

На начальном этапе был проанализирован существующий проект, оставшийся после предыдущих практических заданий. Изначально приложение разворачивалось с помощью набора обычных Kubernetes-манифестов, размещённых в каталоге `k8s/`. В проекте уже присутствовали манифесты для `Namespace`, `ConfigMap`, `Deployment`, `Service`, `Gateway`, `HTTPRoute`, а также отдельное развертывание PostgreSQL на базе `StatefulSet`.

Структура приложения включала:

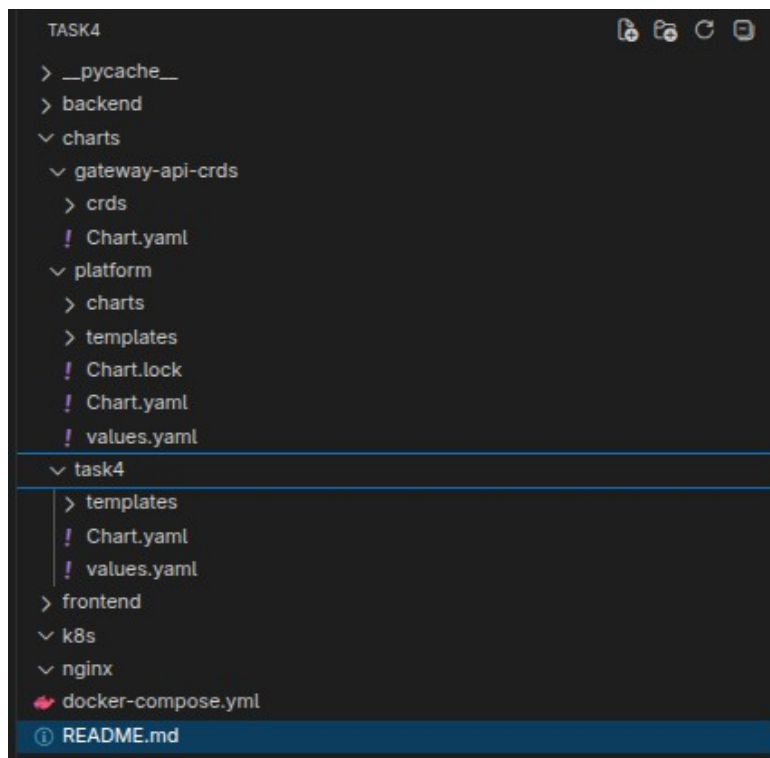
- backend/ — серверную часть на Flask, работающую с PostgreSQL;
- frontend/ — клиентскую часть, отдающую статическую HTML-страницу и обращающуюся к backend API;
- docker-compose.yml — конфигурацию для локального запуска;
- k8s/ — Kubernetes-манифесты для развертывания в кластере.

На этом этапе была поставлена задача отказаться от управления приложением через отдельные YAML-манифесты и перейти к Helm-чартам. Такой подход удобнее для сопровождения, повторного использования конфигурации и параметризации развертывания. Кроме того, использование Helm позволяет централизованно управлять зависимостями и упрощает установку сложных компонентов, таких как `cert-manager`, `CloudNativePG` и `NGINX Gateway Fabric`.

В результате было принято решение разделить развертывание на два независимых Helm-чарта:

- `charts/platform` — для платформенных компонентов кластера;
- `charts/task4` — для приложения и связанных с ним ресурсов.

Такое разделение позволило отделить инфраструктуру уровня кластера от прикладного уровня. Это особенно важно, поскольку `cert-manager`, `CloudNativePG` и контроллер `Gateway API` относятся к системным компонентам и не должны смешиваться с шаблонами самого приложения.



Итоговая структура проекта

Создание Helm-структуры и перенос манифестов приложения в chart task4

После анализа исходного проекта была создана новая Helm-структура, в которой все ресурсы приложения были перенесены из обычных Kubernetes-манифестов в шаблоны Helm. Для этого в каталоге `charts/` был создан chart `task4`, предназначенный для установки самого приложения и связанных с ним прикладных ресурсов.

В состав chart task4 были включены:

- Deployment и Service для backend;
- Deployment и Service для frontend;
- ConfigMap с параметрами приложения;
- ресурсы Gateway и HTTPRoute;
- TLS-ресурсы для cert-manager;
- ресурс Cluster оператора CloudNativePG;
- секрет с учетными данными приложения для подключения к базе данных.

На этом этапе была выполнена параметризация шаблонов через файл `values.yaml`. В него были вынесены:

- имена Docker-образов frontend и backend;
- число реплик;
- порты сервисов;

- параметры доступа к PostgreSQL;
- настройки Gateway API;
- имя хоста task4.local;
- параметры TLS-сертификатов;
- параметры создаваемого CloudNativePG кластера.

Такой подход позволил отказаться от жёстко зашитых значений внутри манифестов и сделать развертывание гибким. В дальнейшем это упрощает повторное использование chart-а и позволяет настраивать установку через `--set` или отдельные values-файлы.

Особое внимание было уделено логике подключения backend к базе данных. В старой схеме приложение подключалось к самостоятельно развернутому PostgreSQL-сервису. После перехода на `CloudNativePG` backend был перенастроен на использование read-write сервиса `-rw`, автоматически создаваемого оператором для доступа к основному экземпляру базы данных.

Также на этом этапе были добавлены Helm helper-шаблоны для единообразного формирования имён ресурсов, меток и селекторов. Это позволило упростить шаблоны и сделать структуру chart-а более аккуратной и поддерживаемой.

Корректность chart-а `task4` была проверена командами:

- helm lint charts/task4
- helm template task4 charts/task4 -n task4

В результате был получен рабочий Helm-chart приложения, который рендерил все необходимые Kubernetes-ресурсы и стал основой дальнейшего развертывания в кластере.

```
[maksimka@archlinux task4]$ ls
backend charts docker-compose.yml frontend k8s nginx __pycache__ README.md
[maksimka@archlinux task4]$ helm lint charts/task4
==> Linting charts/task4
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
[maksimka@archlinux task4]$
```

Результат проверки chart-а task4

```
[maksimka@archlinux task4]$ helm template task4 charts/task4 -n task4
---
# Source: task4/templates/cnpg-app-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: task4-db-app-auth
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
type: kubernetes.io/basic-auth
stringData:
  username: "task4user"
  password: "task4password"
---
# Source: task4/templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: task4-task4-config
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
data:
  API_BASE_URL: "/api"
  POSTGRES_HOST: "task4-db-rw"
  POSTGRES_PORT: "5432"
  POSTGRES_DB: "task4db"
---
# Source: task4/templates/backend.yaml
```

Сгенерированные ресурсы Secret и ConfigMap chart-a task4

```

# Source: task4/templates/backend.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task4-task4-backend
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/component: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app.kubernetes.io/name: task4
      app.kubernetes.io/instance: task4
      app.kubernetes.io/component: backend
  template:
    metadata:
      labels:
        app.kubernetes.io/name: task4
        app.kubernetes.io/instance: task4
        app.kubernetes.io/component: backend
    spec:
      containers:
        - name: backend
          image: "docker.io/mozlhlgh/task4-backend:latest"
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
              name: http
          env:
            - name: POSTGRES_HOST
              valueFrom:
                configMapKeyRef:
                  name: task4-task4-config
                  key: POSTGRES_HOST
            - name: POSTGRES_PORT
              valueFrom:
                configMapKeyRef:
                  name: task4-task4-config
                  key: POSTGRES_PORT
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef:
                  name: task4-task4-config
                  key: POSTGRES_DB
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: task4-db-app-auth
                  key: username
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: task4-db-app-auth
                  key: password
            - name: POSTGRES_CONNECT_RETRIES
              value: "2"
            - name: POSTGRES_CONNECT_DELAY
              value: "1"
          startupProbe:
            httpGet:
              path: /startup
              port: http
            failureThreshold: 30
            periodSeconds: 5
            timeoutSeconds: 30
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            initialDelaySeconds: 5
            periodSeconds: 10
            timeoutSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 10
            periodSeconds: 20

```

Сгенерированный Deployment backend-компонента

```

# Source: task4/templates/gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: task4-task4-gateway
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
spec:
  gatewayClassName: nginx
  listeners:
    - name: http
      protocol: HTTP
      port: 80
      hostname: "task4.local"
      allowedRoutes:
        namespaces:
          from: Same
    - name: https
      protocol: HTTPS
      port: 443
      hostname: "task4.local"
      tls:
        mode: Terminate
        certificateRefs:
          - kind: Secret
            name: task4-gateway-tls
      allowedRoutes:
        namespaces:
          from: Same
---
# Source: task4/templates/gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: task4-task4-redirect
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
spec:
  hostnames:
    - "task4.local"
  parentRefs:
    - name: task4-task4-gateway
      sectionName: http
  rules:
    - filters:
        - type: RequestRedirect
          requestRedirect:
            scheme: https
            statusCode: 301
---
# Source: task4/templates/gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: task4-task4-route
  labels:
    app.kubernetes.io/name: task4
    helm.sh/chart: task4-0.1.0
    app.kubernetes.io/instance: task4
    app.kubernetes.io/managed-by: Helm
spec:
  hostnames:
    - "task4.local"
  parentRefs:
    - name: task4-task4-gateway
      sectionName: https
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: task4-task4-backend
          port: 8080
    - matches:
        - path:

```

Сгенерированные ресурсы Gateway и HTTPRoute

Создание chart `platform` и установка инфраструктурных компонентов через Helm

После подготовки chart-a task4 следующим этапом стало создание отдельного Helm chart-а для инфраструктурных компонентов кластера. Для этого был создан chart `platform`, предназначенный для установки системных зависимостей, необходимых для работы приложения.

Выделение этих компонентов в отдельный chart было необходимо по нескольким причинам. Во-первых, такие решения, как `cert-manager`, CloudNativePG и NGINX Gateway Fabric, относятся не к самому приложению, а к инфраструктуре Kubernetes-кластера. Во-вторых, отдельная установка платформенных и прикладных компонентов делает развертывание более понятным и удобным для сопровождения. В-третьих, такой подход позволяет переиспользовать platform-уровень и для других приложений при необходимости.

В chart platform были включены следующие зависимости:

- cert-manager — для выпуска и управления TLS-сертификатами;
- CloudNativePG — для операторного управления PostgreSQL;
- NGINX Gateway Fabric — в качестве контроллера Gateway API;
- Gateway API CRDs — как отдельный локальный Helm dependency chart.

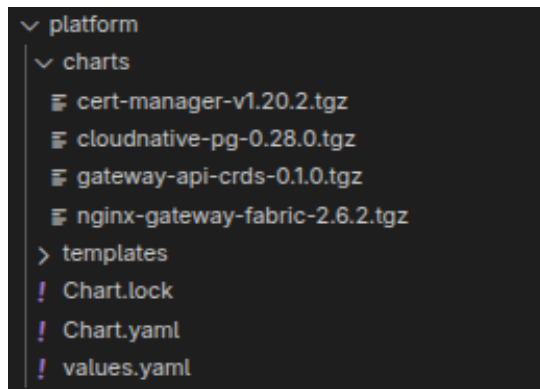
Для Gateway API был создан локальный chart `gateway-api-crds`, в который были добавлены официальные CRD стандартного канала. Это было необходимо для выполнения требования установки Gateway API через Helm, а не вручную через kubectl apply. В результате все основные платформенные компоненты также стали устанавливаться единым Helm-способом.

Внутри platform chart-а были определены зависимости в Chart.yaml, а основные параметры вынесены в values.yaml. В конфигурации были заданы:

- включение cert-manager;
- включение CloudNativePG;
- включение NGINX Gateway Fabric;
- использование типа сервиса NodePort для gateway data plane;
- ограничение наблюдаемых namespace значением task4;
- версия стандартного набора Gateway API CRDs.

После подготовки структуры chart-а была выполнена проверка зависимостей и корректности рендеринга. Это позволило убедиться, что все внешние компоненты могут быть установлены как единый platform-слой до развертывания самого приложения.

Таким образом, на данном этапе была сформирована отдельная инфраструктурная часть проекта, которая отвечает за маршрутизацию трафика, управление PostgreSQL и выпуск TLS-сертификатов в Kubernetes-кластере.



Структура Helm chart-a platform

```

apiVersion: v2
name: platform
description: Helm chart for cluster-level platform components used by task4
type: application
version: 0.1.0
appVersion: "1.0.0"
dependencies:
- name: cert-manager
  alias: certmanager
  repository: oci://quay.io/jetstack/charts
  version: v1.20.2
  condition: certmanager.enabled
- name: cloudnative-pg
  alias: cloudnativepg
  repository: https://cloudnative-pg.github.io/charts
  version: 0.28.0
  condition: cloudnativepg.enabled
- name: nginx-gateway-fabric
  alias: nginxgatewayfabric
  repository: oci://ghcr.io/nginx/charts
  version: 2.6.2
  condition: nginxgatewayfabric.enabled
- name: gateway-api-crds
  repository: file:///./gateway-api-crds
  version: 0.1.0
  condition: gatewayApi.enabled

```

Файл Chart.yaml chart-a platform с описанием зависимостей

```

certmanager:
  enabled: true
  crds:
    enabled: true
  prometheus:
    enabled: false

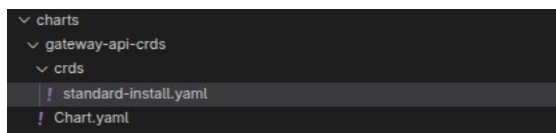
cloudnativepg:
  enabled: true
  nameOverride: cloudnative-pg
  config:
    create: true
    name: cnpg-controller-manager-config
    data: {}

nginxgatewayfabric:
  enabled: true
  nginx:
    service:
      type: NodePort
  nginxGateway:
    watchNamespaces:
      - task4

gatewayApi:
  enabled: true
  standardChannelVersion: v1.4.1

```

Файл values.yaml chart-a platform



Локальный chart gateway-api-crds с CRD для Gateway API

Проверка и установка chart-a platform

После подготовки Helm chart-a platform следующим этапом стала проверка его корректности и установка инфраструктурных компонентов в Kubernetes-кластер. На этом шаге было необходимо убедиться, что chart корректно собирает зависимости и может быть использован для развертывания системных компонентов.

Сначала была выполнена загрузка Helm-зависимостей, описанных в Chart.yaml. Для этого использовалась команда `helm dependency update charts/platform`. В результате в каталог `charts/platform/charts/` были загружены внешние chart-ы `cert-manager`, `CloudNativePG`, `NGINX Gateway Fabric`, а также локально подключённый chart `'gateway-api-crds'`.

После загрузки зависимостей была выполнена проверка chart-a командой `'helm lint charts/platform'`. Эта проверка позволила убедиться в корректности структуры chart-a и отсутствии критических ошибок в его конфигурации. Далее был выполнен рендер итоговых манифестов с помощью команды `'helm template'`, чтобы проверить, какие ресурсы будут созданы в результате установки.

Установка platform выполнялась в отдельный namespace `platform-system`, поскольку этот chart предназначен для системных компонентов кластера. Перед установкой также был создан namespace `task4`, так как `NGINX Gateway Fabric` настраивался на наблюдение только за этим namespace.

При установке chart-a использовался отдельный `GatewayClass` с именем `'task4-nginx'`. Это было необходимо для того, чтобы не конфликтовать с уже существующим в кластере `'GatewayClass nginx'`, оставшимся от предыдущих развертываний. Таким образом, новый gateway-контроллер получил собственный класс и мог работать независимо от ранее созданных ресурсов.

Базовая команда установки platform chart-a была дополнена параметрами `'--set'`, позволяющими переопределить отдельные настройки при установке. В частности, таким способом было задано имя `'GatewayClass'`, используемого контроллером `'NGINX Gateway Fabric'`.

После успешной установки chart-a были проверены:

- наличие CRD `'Gateway API'`;
- наличие развернутых platform-компонентов в namespace `'platform-system'`;
- создание `GatewayClass`;
- корректная работа gateway-контроллера, `cert-manager` и `CloudNativePG`.

```
[maksimka@archlinux task4]$ kubectl get pods -n platform-system
```

NAME	READY	STATUS	RESTARTS	AGE
platform-certmanager-cainjector-6d677fdd44-lj4m7	1/1	Running	2 (60m ago)	8d
platform-certmanager-d7d64ccc4-dfwtk	1/1	Running	2 (60m ago)	8d
platform-certmanager-webhook-d8b8cb6b6-67nl6	1/1	Running	2 (60m ago)	8d
platform-cloudnative-pg-6d8fb756df-2dxx8	1/1	Running	3 (60m ago)	8d
platform-nginxgatewayfabric-6fdb88d8dc-xdd5k	1/1	Running	2 (60m ago)	8d

Состояние pod'ов после установки chart-a platform

Развертывание приложения task4 в Kubernetes

После установки platform-компонентов было выполнено развертывание самого приложения с помощью Helm chart-a task4. Установка производилась в namespace task4, а в качестве GatewayClass использовалось имя `task4-nginx`, соответствующее ранее установленному gateway-контроллеру.

Перед запуском базы данных было учтено, что в используемом кластере отсутствует StorageClass. По этой причине для CloudNativePG был дополнительно создан статический PersistentVolume, который использовался для хранения данных PostgreSQL.

После этого был выполнен Helm install приложения, в результате которого в кластере были созданы:

- frontend и backend компоненты приложения;
- CloudNativePG Cluster для PostgreSQL;
- Gateway и HTTPRoute;
- TLS-ресурсы cert-manager.

После завершения установки были проверены pod'ы и сервисы в namespace `task4`. В результате приложение было успешно развернуто и стало доступно внутри кластера, а также подготовлено к внешней проверке через Gateway API.

```
[maksimka@archlinux task4]$ kubectl get pods -n task4
```

NAME	READY	STATUS	RESTARTS	AGE
task4-db-1	1/1	Running	2 (64m ago)	8d
task4-task4-backend-5d47f77f67-ppxqz	1/1	Running	1 (74m ago)	8d
task4-task4-frontend-7f987fd447-kqrk7	1/1	Running	1 (74m ago)	8d
task4-task4-gateway-task4-nginx-7bd98c8cc7-hxsd8	1/1	Running	2 (64m ago)	8d

Состояние pod'ов приложения в namespace task4

```
[maksimka@archlinux task4]$ kubectl get svc -n task4
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
task4-db-rw	ClusterIP	10.111.253.121	<none>	5432/TCP	8d
task4-task4-backend	ClusterIP	10.102.228.213	<none>	8080/TCP	8d
task4-task4-frontend	ClusterIP	10.110.189.1	<none>	80/TCP	8d
task4-task4-gateway-task4-nginx	NodePort	10.101.68.167	<none>	80:30441/TCP, 443:31073/TCP	8d

Список сервисов приложения в namespace task4

Проверка работоспособности приложения

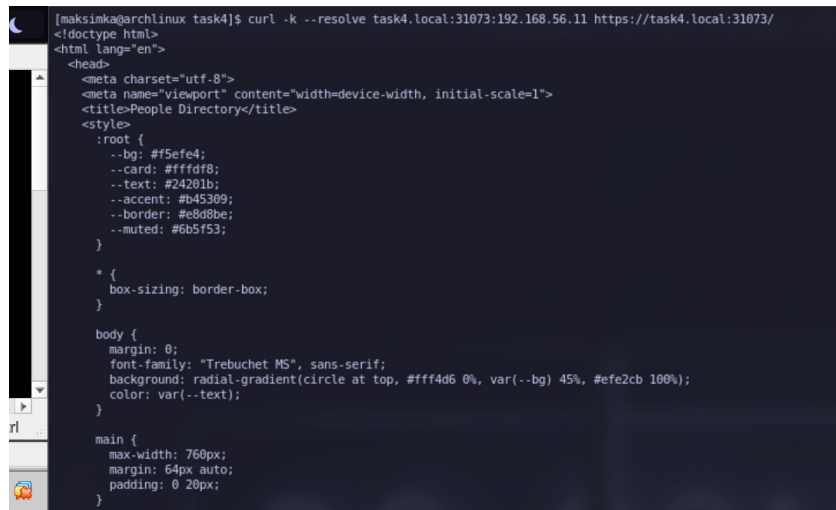
После завершения развертывания была выполнена проверка доступности приложения и корректности маршрутизации через Gateway API. Для этого были определены IP-адрес ноды Kubernetes и NodePort, назначенные gateway-сервису.

Проверка выполнялась с помощью `curl` по HTTP и HTTPS. При обращении по HTTP приложение возвращало ответ `301 Moved Permanently`, что соответствовало настроенному редиректу с HTTP на HTTPS. При обращении по HTTPS с использованием hostname `task4.local` frontend корректно открывался, а backend API успешно возвращал список записей из PostgreSQL.

Дополнительно была проверена работа метода создания новой записи через `POST /api/people/random`. После выполнения запроса данные успешно добавлялись в базу, что подтверждало корректную работу backend, gateway и PostgreSQL.

Также приложение было открыто в браузере по адресу `https://task4.local:<HTTPS\_NODEPORT>/`. Поскольку использовался self-signed сертификат, браузер выводил предупреждение безопасности, что является ожидаемым поведением в рамках учебного стенда.

Таким образом, было подтверждено, что приложение успешно работает в Kubernetes-кластере, доступно через Gateway API по HTTPS и использует PostgreSQL, управляемый оператором CloudNativePG.



```
[maksimka@archlinux task4]$ curl -k --resolve task4.local:31073:192.168.56.11 https://task4.local:31073/
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>People Directory</title>
<style>
:root {
  --bg: #f5efe4;
  --card: #fffff8;
  --text: #24201b;
  --accent: #b45309;
  --border: #e8d8be;
  --muted: #6b5f53;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Trebuchet MS", sans-serif;
  background: radial-gradient(circle at top, #fff4d6 0%, var(--bg) 45%, #efe2cb 100%);
  color: var(--text);
}

main {
  max-width: 760px;
  margin: 64px auto;
  padding: 0 20px;
}
```

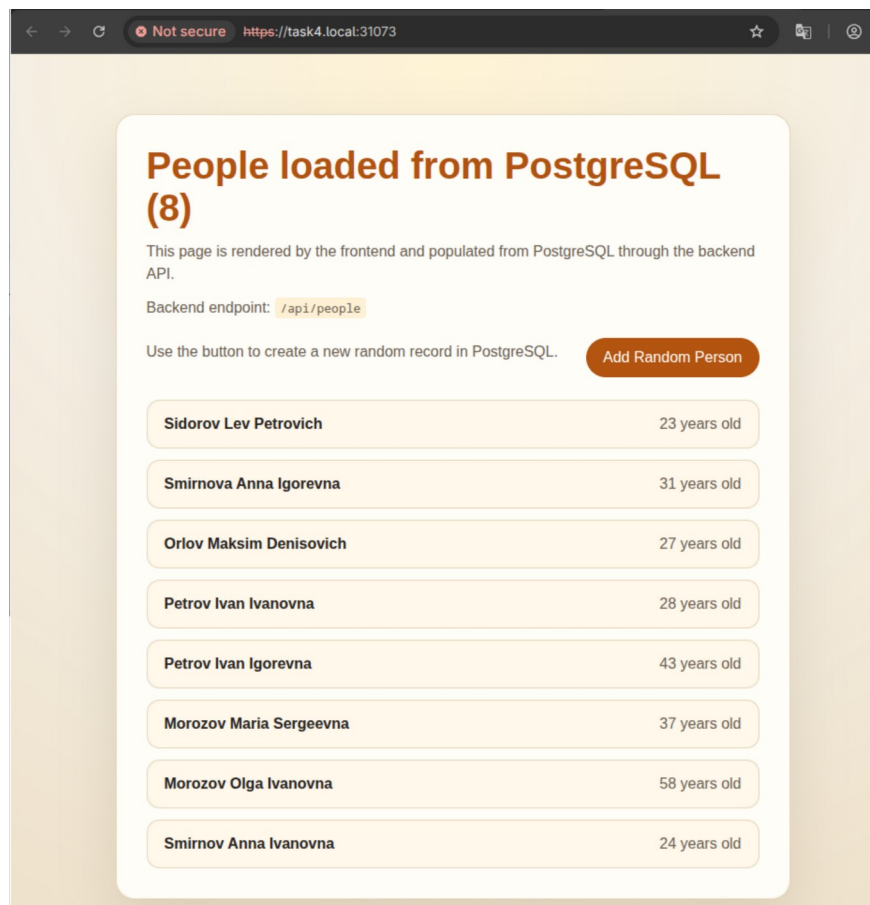
Проверка главной страницы приложения по HTTPS

```
[maksimka@archlinux task4]$ curl -k --resolve task4.local:31073:192.168.56.11 https://task4.local:31073/api/people
{"count":7,"message":"People loaded from PostgreSQL","people":[{"age":23,"first_name":"Lev","id":1,"last_name":"Sidorov","middle_name":"Petrovich"}, {"age":31,"first_name":"Anna","id":2,"last_name":"Smirnova","middle_name":"Igorevna"}, {"age":27,"first_name":"Maksim","id":3,"last_name":"Orlov","middle_name":"Denisovich"}, {"age":28,"first_name":"Ivan","id":4,"last_name":"Petrov","middle_name":"Ivanovna"}, {"age":43,"first_name":"Ivan","id":5,"last_name":"Petrov","middle_name":"Igorevna"}, {"age":37,"first_name":"Maria","id":6,"last_name":"Morozov","middle_name":"Sergeevna"}, {"age":58,"first_name":"Olga","id":7,"last_name":"Morozov","middle_name":"Ivanovna"}]}
[maksimka@archlinux task4]$
```

Проверка backend API по адресу /api/people

```
[maksimka@archlinux task4]$ curl -k -X POST --resolve task4.local:31073:192.168.56.11 https://task4.local:31073/api/people/random
{"message":"Random person added to PostgreSQL","person":{"age":24,"first_name":"Anna","id":8,"last_name":"Smirnov","middle_name":"Ivanovna"}}
[maksimka@archlinux task4]$
```

Проверка добавления новой записи через POST /api/people/random



Проверка через браузер

Выводы

В ходе выполнения работы было выполнено полное переработанное развертывание приложения в Kubernetes с переходом от обычных манифестов к Helm chart-ам. В результате проект получил более удобную и структурированную схему установки, разделённую на platform- и application-уровень.

В рамках работы были решены все поставленные задачи:

- приложение было переведено на Helm;
- Gateway API был установлен через Helm;
- самостоятельно развернутый PostgreSQL был заменён на оператор CloudNativePG;
- для HTTPS были добавлены self-signed сертификаты с использованием cert-manager.

Также была выполнена настройка маршрутизации через `NGINX Gateway Fabric`, обеспечен доступ к приложению по HTTPS и проверена работа frontend, backend и базы данных PostgreSQL в Kubernetes-кластере.

В процессе выполнения были закреплены практические навыки:

- создания и настройки Helm chart-ов;
- работы с Gateway API;
- использования операторного подхода для управления PostgreSQL;
- настройки TLS-сертификатов в Kubernetes;
- проверки и тестирования развернутого приложения.

Итогом работы стало полностью функционирующее приложение, доступное через Gateway API по HTTPS и использующее PostgreSQL, управляемый оператором CloudNativePG.